

浙江大学

本科实验报告

课程名称：电子电路设计实验 II

姓 名：

学 院：信息与工程学院

专 业：信息工程

学 号：

指导教师：叶险峰，施红军，邓靖靖

2026 年 5 月 11 日

浙江大学实验报告

课程名称：_____电子电路设计实验 II _____指导老师：__叶险峰，施红军，邓靖靖__成绩：_____
实验名称：_A6、Arduino 多功能数字时钟的设计与制作_实验类型：_设计型_同组学生姓名：__

一、示例项目设计过程

1) 设计任务与要求

设计电流-电压转换电路，将 4~20mA 标准电流信号转换为 0~10V 的标准电压信号。要求：

①参考图 1 的电流-电压转换电路电路图，确定各元件值。

②用 Altium Designer 软件设计原理图和 PCB 图。

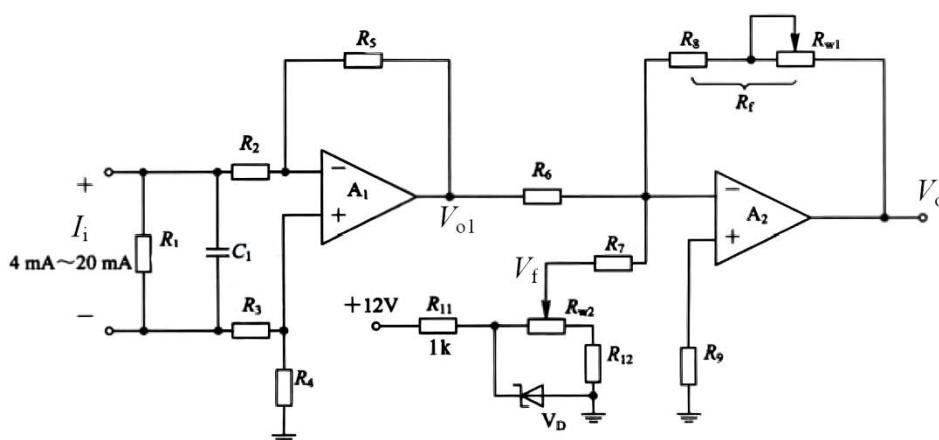
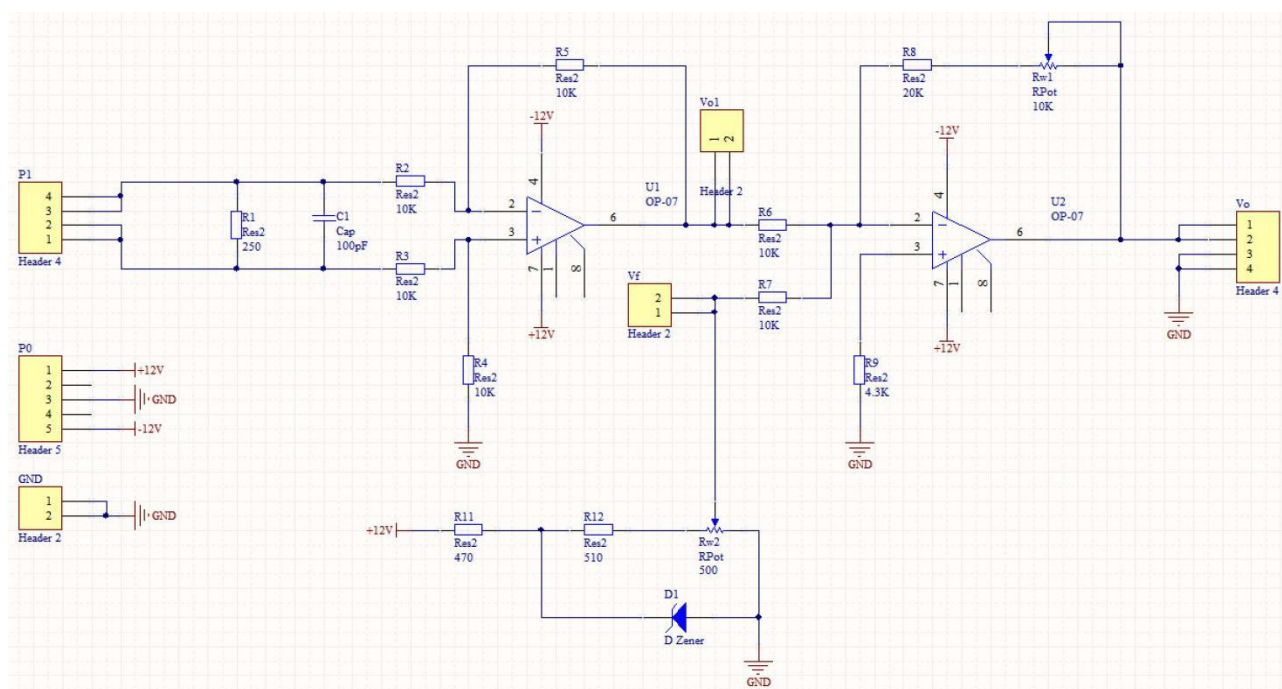


图 1 示例项目电路图

2) 原理图和 PCB 图设计结果

依据图 1，我利用 Altium Designer 软件设计了原理图和 PCB 图，分别如图 2 和图 3 所示。



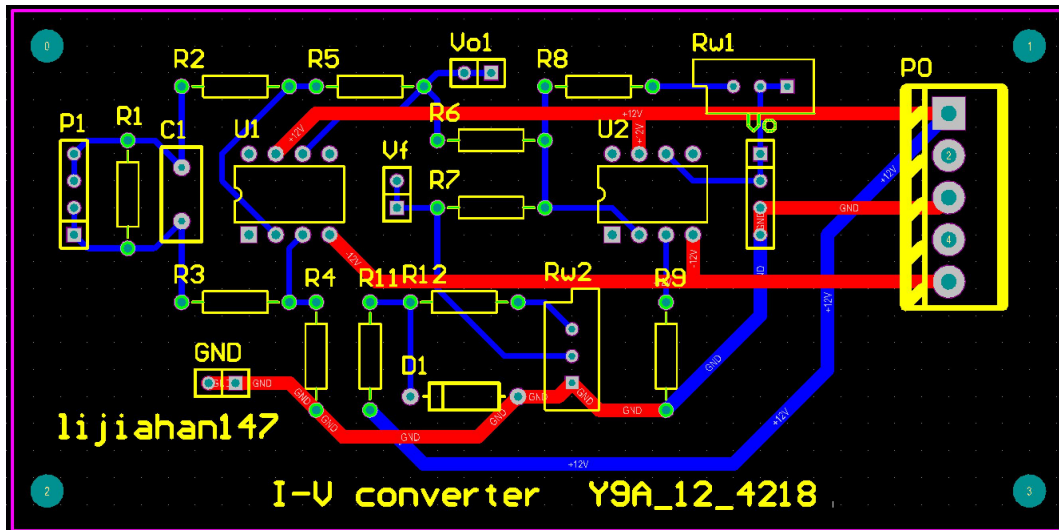


图3 示例项目设计 PCB 图

3) 元器件焊接

依据 PCB 图制作的电路板如图 4 所示，元器件焊接完成后的电路板如图 5 所示。

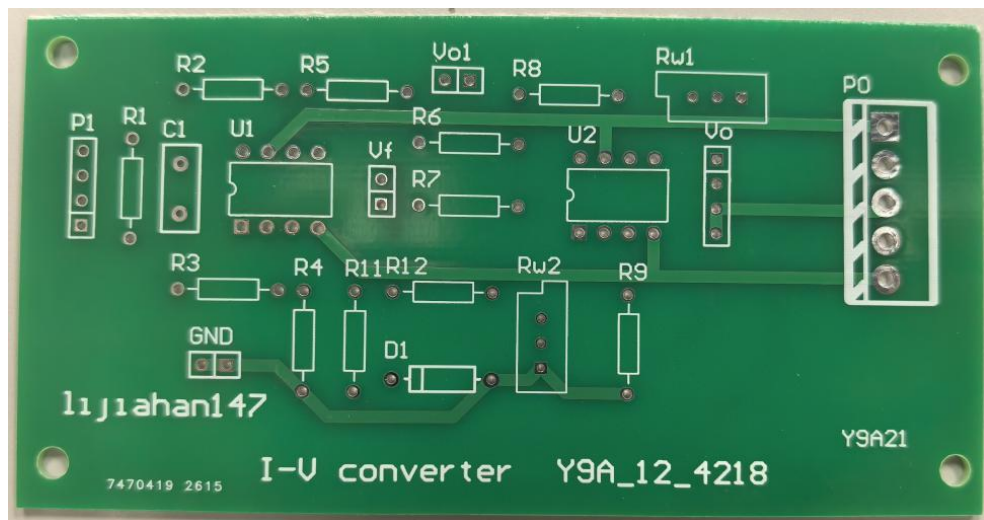


图4 示例项目电路板（未焊接）

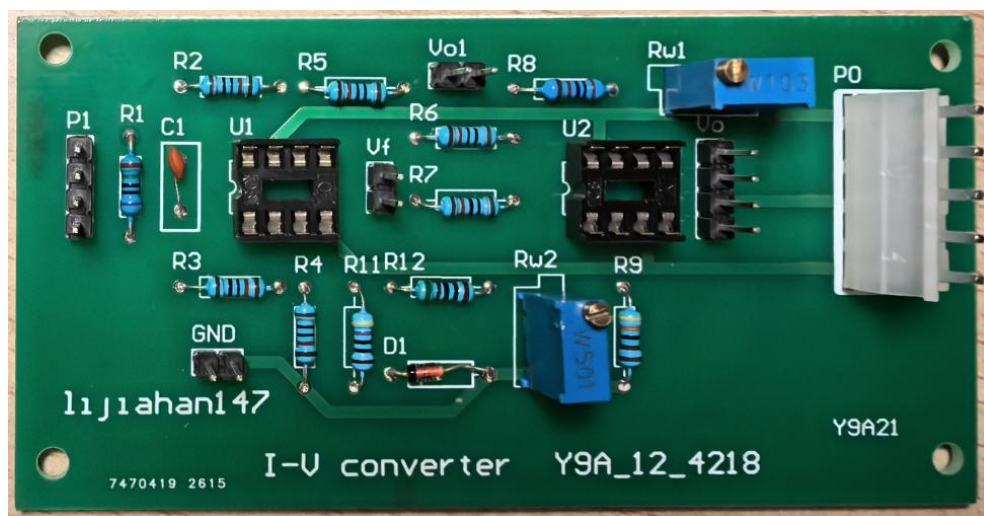


图5 示例项目电路板（已焊接）

4) 电路板调试

对焊接完成的电路板进行测试，观察其电流-电压转换情况，结果如表 1 所示。

电流/mA	电压测试值/V	电压理论值/V	相对误差
4	0.00	0	0.00%
5	0.63	0.625	0.80%
6	1.26	1.25	0.80%
7	1.88	1.875	0.27%
8	2.50	2.5	0.00%
9	3.13	3.125	0.16%
10	3.75	3.75	0.00%
11	4.37	4.375	0.11%
12	4.99	5	0.20%
13	5.62	5.625	0.09%
14	6.24	6.25	0.16%
15	6.86	6.875	0.22%
16	7.49	7.5	0.13%
17	8.11	8.125	0.18%
18	8.73	8.75	0.23%
19	9.35	9.375	0.27%
20	9.98	10	0.20%

表 1 示例项目电路板测试结果

5) 实验结论

从表 1 可以看出，各测试点电压测试值与理论值非常接近，说明该电路板实现了电流-电压转换的功能，本次示例项目成功。

二、实验目的

- 1) 学习掌握用 Arduino UNO 设计数字时钟；
- 2) 学习掌握 PCB 电路板的设计和制作；
- 3) 学习掌握 Arduino UNO 扩展板的设计与制作；
- 4) 学习掌握 DS1302 时钟芯片和 LCD1602 液晶显示屏的使用。

三、实验任务与要求

- 1) 用 Arduino UNO 设计多功能数字时钟，要求实现功能：
 - ①显示时间、日期和星期
 - ②断电保存时间
 - ③通过按钮设置时间、日期
 - ④整点响铃
 - ⑤自定义闹钟
 - ⑥显示温度
 - ⑦自定义报警温度
 - ⑧按键功能：按选择键进入设置时间功能；同时按 +- 键进入闹钟和报警温度设置功能；
 - ⑨再按选择键光标跳动，光标跳到哪，当前的参数即可通过加减键修改。
- 2) 设计的电路，完成相应器件的选择和参数计算，制作 Arduino UNO 扩展板。
- 3) 编制与调试多功能数字时钟程序。

4) 将制作的 Arduino UNO 扩展板与 Arduino UNO 板组装后, 进行系统联调。

四、实验原理

本项目使用 Arduino UNO 设计多功能数字时钟。硬件框图如图 6 所示, 需要采用 DS1302 时钟芯片、LCD 显示模块、LM35 温度传感器等模块, 参考电路图如图 7 所示。下面对各部分模块进行说明。

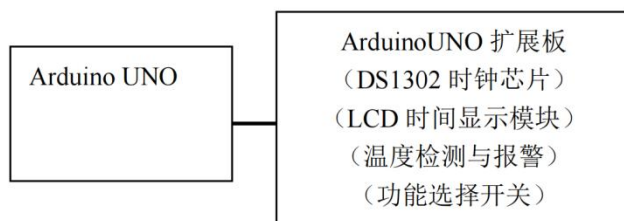


图 6 设计项目硬件框图

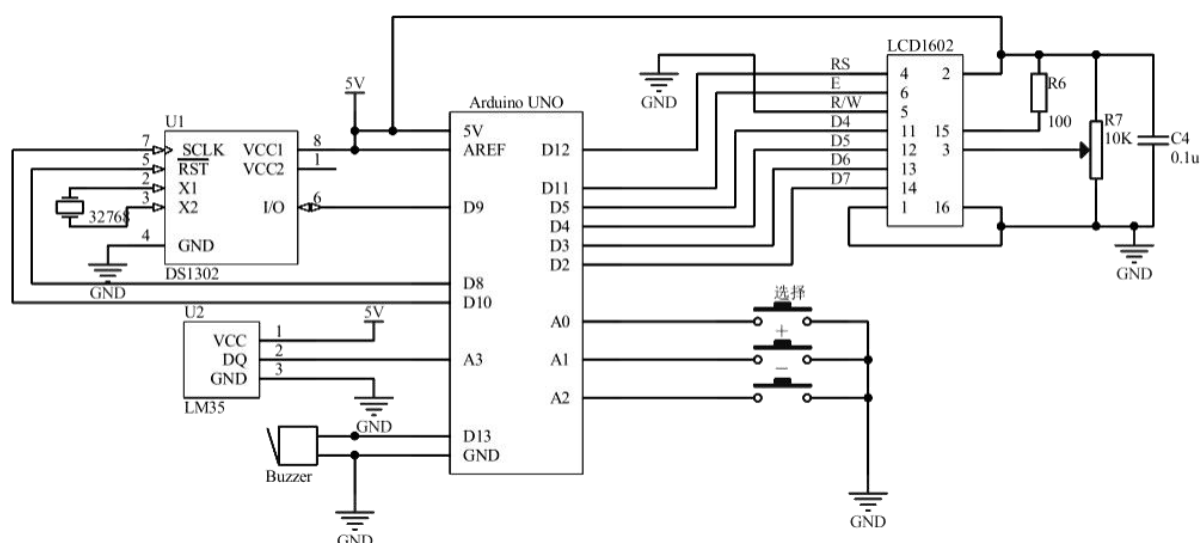


图 7 设计项目电路图

DS1302 时钟芯片

DS1302 是美国 DALLAS 公司推出的一种高性能、低功耗的实时时钟芯片, 附加 31 字节静态 RAM, 采用 SPI 三线接口与 CPU 进行同步通信, 并可采用突发方式一次传送多个字节的时钟信号和 RAM 数据。实时时钟可提供秒、分、时、日、星期、月和年, 一个月小与 31 天时可以自动调整, 且具有闰年补偿功能。工作电压宽达 2.5~5.5V。采用双电源供电 (主电源和备用电源), 可设置备用电源充电方式, 提供了对后背电源进行涓细电流充电的能力。

LCD 显示模块

本次实验使用 Arduino UNO 直接驱动 1602 液晶显示字母和数字, LCD 显示模块有一个优势就是将驱动电路集成在模块中。LCD 显示模块具有标准, 不同的生产商所生产的大多数模块都可以按相同方法来使用。1602 液晶在应用中非常广泛, 最初的 1602 液晶使用的是 HD44780 控制器, 现在各个厂家的 1602 模块基本上都是采用了与之兼容的 IC, 所以特性上基本都是一致的。

温度传感器

温度传感器可采用 LM35。LM35 是美国国家半导体公司生产的, 具有很高的工作精度和较宽的线性工作范围, 工作电压范围为 4~30V。其精度为 0.5°C, 输出电压与摄氏温度成线性比例关系, 非线性温度误差低于 $\pm 0.25^{\circ}\text{C}$, 线性温度系数为 $+10\text{mV}/^{\circ}\text{C}$, 无需外部校准或微调。LM35 的供电模式有单电源与正负双电源两种模式。正负双电源供电模式可提供 $-55^{\circ}\text{C}\sim 150^{\circ}\text{C}$ 的全量程范围测量; 单电源供电模式在 25°C 下电流约为 $50\mu\text{A}$, 温度测量范围 $0^{\circ}\text{C}\sim 100^{\circ}\text{C}$, 其输出电压范围为 0~1V。

五、实验方案设计与参数计算

1) 硬件设计

依据图 7，我利用 Altium Designer 软件，结合实际的 Arduino UNO 板和 1602 液晶，设计了原理图和 PCB 图，分别如图 8 和图 9 所示。

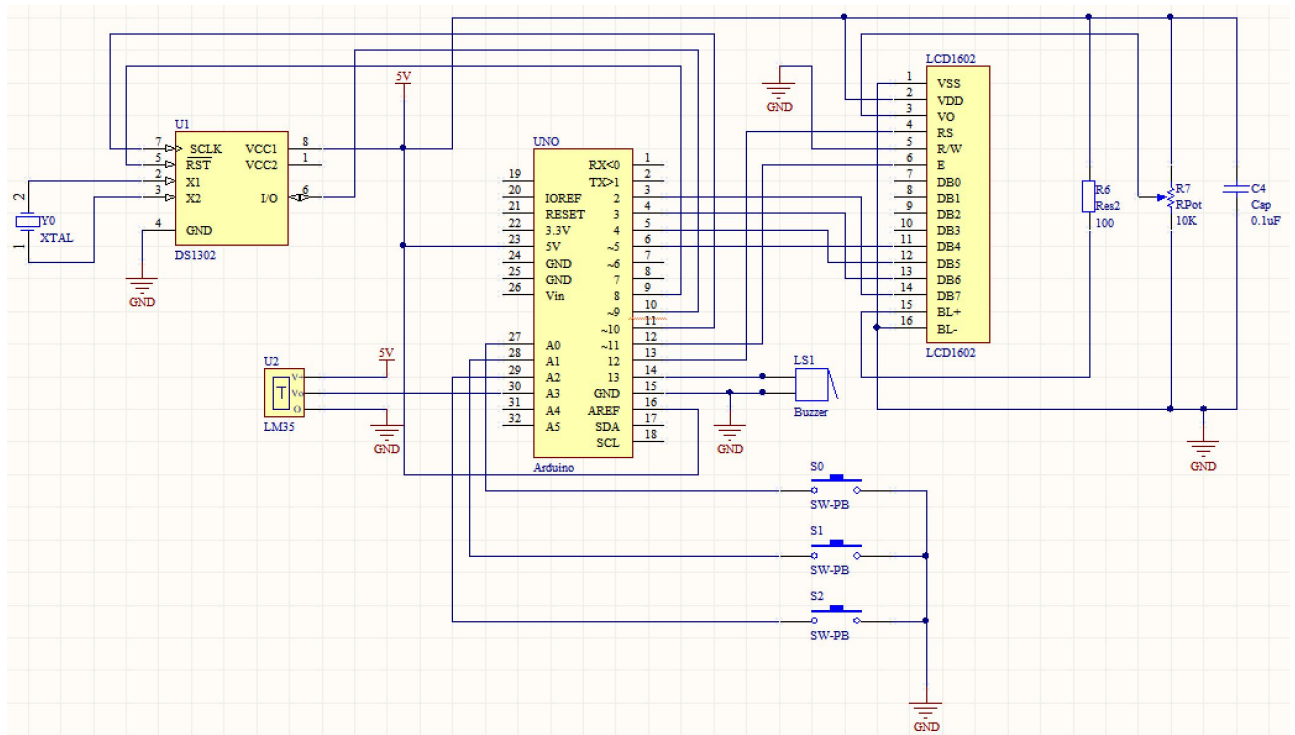


图 8 设计项目设计原理图

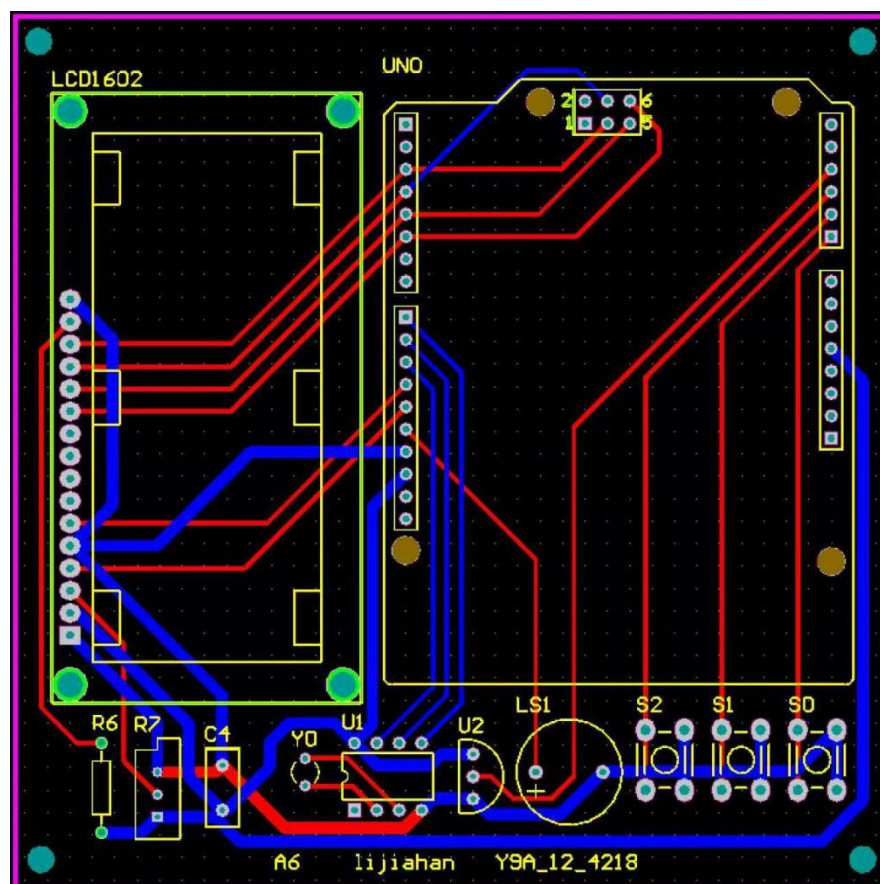


图 9 设计项目设计 PCB 图

2) 软件设计

基于参考代码，我进行了一些逻辑优化和结构调整，具体为新增了按键蜂鸣功能，整点蜂鸣蜂鸣次数改为对应的小时数，闹钟铃声改为《小星星》并显示“GET UP”，超过指定温度报警改为连续蜂鸣警报并显示“TOO HOT”，最终的程序如下。

```
/* *
 * LCD RS pin to digital pin 12
 * LCD Enable pin to digital pin 11
 * LCD D4 pin to digital pin 5
 * LCD D5 pin to digital pin 4
 * LCD D6 pin to digital pin 3
 * LCD D7 pin to digital pin 2
 * LCD R/W pin to ground
 * LCD VSS pin to ground
 * LCD VCC pin to 5V
 * */

#include <DS1302.h>
#include <LiquidCrystal.h>          //LCD1602 显示头文件

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
#define choose  A0    //选择端口
#define add     A1     //加
#define minus   A2     //减
#define Tone 13        //蜂鸣器端口
uint8_t CE_PIN   = 8;   //DS1302 RST 端口
uint8_t IO_PIN   = 9;   //DS1302 DAT 端口
uint8_t SCLK_PIN = 10;  //DS1302 CLK 端口
DS1302 rtc(CE_PIN, IO_PIN, SCLK_PIN); //创建 DS1302 对象
unsigned long seconds;
int s = 0, m = 0, h = 0, d = 0, mon = 0, y = 0;    //时间进位
int second = 0, minute = 0, hour = 0, day = 0, month = 0, year = 0; //当前时间
int SECOND = 0, MINUTE = 0, HOUR = 0, DAY = 0, MONTH = 0, YEAR = 0; //初始时间
int chose = 0, alarm_choose = 0 ,ButtonDelay = 10, frequency0 = 2093;
int frequency1 = 523,frequency2 = 587,frequency3 = 659,frequency4 = 698,frequency5 =
784,frequency6 = 880,frequency7 = 988;
int alarm_hour = 7, alarm_minute = 30, alarm_second = 0; //闹钟时间
float Temperatures, Temp_Alarm = 30 ;

/** 格式化输出 */
void FormatDisplay(int col, int row,int num){
    lcd.setCursor(col, row);
    if(num < 10)    lcd.print("0");
    lcd.print(num);
}
```

```

/** 计算时间 */
void time() {
    second = (SECOND + seconds) % 60;    //计算秒
    m = (SECOND + seconds) / 60;         //分钟进位
    FormatDisplay(6,1,second);
    minute = (MINUTE + m) % 60;    //计算分钟
    h = (MINUTE + m) / 60;         //小时进位
    FormatDisplay(3,1,minute);
    hour = (HOUR + h) % 24;    //计算小时
    d = (HOUR + h) / 24;         //天数进位
    FormatDisplay(0,1,hour);
    lcd.setCursor(2, 1);
    lcd.print(":");
    lcd.setCursor(5, 1);
    lcd.print(":");
}
/** 根据年月计算当月天数 */
int Days(int year, int month){
    int days = 0;
    if (month != 2){
        switch(month){
            case 1: case 3: case 5: case 7: case 8: case 10: case 12: days= 31; break;
            case 4: case 6: case 9: case 11: days = 30; break;
        }
    }else{    //闰年
        if(year % 4 == 0 && year % 100 != 0 || year % 400 == 0){
            days = 29;
        }
        else{
            days = 28;
        }
    }
    return days;
}
/** 计算当月天数 */
void Day(){
    int days = Days(year,month);
    int days_up;
    if(month == 1){
        days_up = Days(year - 1, 12);
    }
    else{
        days_up = Days(year, month - 1);
    }
}

```



```

    day = (DAY + d) % days;
    if(day == 0){
        day = days;
    }
    if((DAY + d) == days + 1 ){
        DAY -= days;
        mon++;
    }
    if((DAY + d) == 0){
        DAY += days_up;
        mon--;
    }
    FormatDisplay(8,0,day);
}

/** 计算月份 */
void Month(){
    month = (MONTH + mon) % 12;
    if(month == 0){
        month = 12;
    }
    y = (MONTH + mon - 1) / 12;
    FormatDisplay(5,0,month);
    lcd.setCursor(7, 0);
    lcd.print('-');
}

/** 计算年份 */
void Year(){
    year = ( YEAR + y ) % 9999;
    if(year == 0){
        year = 9999;
    }
    lcd.setCursor(0, 0);
    if(year < 1000){
        lcd.print("0");
    }
    if(year < 100){
        lcd.print("0");
    }
    if(year < 10){
        lcd.print("0");
    }
    lcd.print(year);
    lcd.setCursor(4, 0);
    lcd.print('-');
}

```

```

}
/** 根据年月日计算星期几 */
void Week(int y,int m, int d){
    if(m == 1){
        m = 13;
    }
    if(m == 2){
        m = 14;
    }
    int week = (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)%7+1;
    String weekstr = "";
    switch(week){
        case 1: weekstr = "Mon. "; break;
        case 2: weekstr = "Tues. "; break;
        case 3: weekstr = "Wed. "; break;
        case 4: weekstr = "Thur. "; break;
        case 5: weekstr = "Fri. "; break;
        case 6: weekstr = "Sat. "; break;
        case 7: weekstr = "Sun. "; break;
    }
    lcd.setCursor(11, 0);
    lcd.print(weekstr);
}
/** 显示时间、日期、星期 */
void Display() {
    time();
    Day();
    Month();
    Year();
    Week(year,month,day);
}
/** 显示光标 */
void DisplayCursor(int col, int row) {
    lcd.setCursor(col, row);
    lcd.cursor();
    delay(100);
    lcd.noCursor();
    delay(100);
}
/** 设置初始时间 */
void set(int y, int mon, int d, int h, int m, int s){
    YEAR = y;
    MONTH = mon;
    DAY = d;

```

```

    HOUR = h;
    MINUTE = m;
    SECOND = s;
}
/** 通过按键设置时间 */
void Set_Time(int rol, int row, int &Time){
    DisplayCursor(rol, row);
    if(digitalRead(add) == LOW){
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            Time ++;
            tone(Tone,frequency0);    //按键蜂鸣
            delay(100);
            noTone(Tone);
        }
        Display();
    }
    if(digitalRead(minus) == LOW){
        delay(ButtonDelay);
        if(digitalRead(minus) == LOW){
            Time --;
            tone(Tone,frequency0);    //按键蜂鸣
            delay(100);
            noTone(Tone);
        }
        Display();
    }
}
/** 按键选择 */
void Set_Clock(){
    if(digitalRead(choose)==LOW){
        lcd.setCursor(9, 1);
        lcd.print("SetTime");
        while(1){
            if(digitalRead(choose) == LOW){
                delay(ButtonDelay);
                if(digitalRead(choose) ==LOW){
                    chose++;
                    tone(Tone,frequency0);    //按键蜂鸣
                    delay(100);
                    noTone(Tone);
                }
            }
        }
        seconds = millis()/1000;
    }
}

```

```

    Display();
    if(chose == 1){
        Set_Time(1, 1, HOUR);           //SetHour
    }else if(chose == 2){
        Set_Time(4, 1, MINUTE);         //SetMinute
    }else if(chose == 3){
        Set_Time(7, 1, SECOND);         //SetSecond
    }else if(chose == 4){
        Set_Time(9, 0, DAY);             //SetDay
    }else if(chose == 5){
        Set_Time(6, 0, MONTH);          // SetMonth
    }else if(chose == 6){
        Set_Time(3, 0, YEAR);            //SetYear
    }else if(chose >= 7) {
        chose = 0;
        break;
    }
}
}

}

}

/** 设置闹钟小时 */
void Set_Alarm_Hour(){
    DisplayCursor(1, 1);
    if(digitalRead(add) == LOW){
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            alarm_hour ++;
            tone(Tone,frequency0);      //按键蜂鸣
            delay(100);
            noTone(Tone);
            if(alarm_hour == 24){
                alarm_hour = 0;
            }
            FormatDisplay(0,1,alarm_hour);
        }
    }
}

if(digitalRead(minus) == LOW){
    delay(ButtonDelay);
    if(digitalRead(minus) == LOW){
        alarm_hour --;
        tone(Tone,frequency0);          //按键蜂鸣
        delay(100);
        noTone(Tone);
        if(alarm_hour == -1){

```

```

        alarm_hour = 23;
    }
    FormatDisplay(0,1,alarm_hour);
}
}

/** 设置闹钟分钟 */
void Set_Alarm_Minute(){
    DisplayCursor(4, 1);
    if(digitalRead(add) == LOW) {
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            alarm_minute ++;
            tone(Tone,frequency0);    //按键蜂鸣
            delay(100);
            noTone(Tone);
            if(alarm_minute == 60){
                alarm_minute = 0;
            }
            FormatDisplay(3,1,alarm_minute);
        }
    }
    if(digitalRead(minus) == LOW){
        delay(ButtonDelay);
        if(digitalRead(minus) == LOW){
            alarm_minute --;
            tone(Tone,frequency0);    //按键蜂鸣
            delay(100);
            noTone(Tone);
            if(alarm_minute == -1){
                alarm_minute = 59;
            }
            FormatDisplay(3,1,alarm_minute);
        }
    }
}

/** 设置报警温度 */
void Set_Alarm_Temp(){
    DisplayCursor(10, 1);
    if(digitalRead(add) == LOW) {
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            Temp_Alarm ++;
            tone(Tone,frequency0);    //按键蜂鸣

```

```

        delay(100);
        noTone(Tone);
    }
}
if(digitalRead(minus) == LOW){
    delay(ButtonDelay);
    if(digitalRead(minus) == LOW){
        Temp_Alarm --;
        tone(Tone,frequency0);    //按键蜂鸣
        delay(100);
        noTone(Tone);
    }
}
}
}
/** 进入报警设置 */
void Set_Alarm(){
    if(digitalRead(add) == LOW && digitalRead(minus) == LOW){
        alarm_hour = hour;
        alarm_minute = minute;
        //alarm_choose = 1;
        lcd.setCursor(0, 0);
        lcd.print("set alarm      ");
        lcd.setCursor(6, 1);
        lcd.print("00");          //闹钟秒数
        while(1){
            if(digitalRead(choose) == LOW){
                delay(ButtonDelay);
                if(digitalRead(choose) == LOW){
                    alarm_choose++;
                    tone(Tone,frequency0);    //按键蜂鸣
                    delay(100);
                    noTone(Tone);
                }
            }
            lcd.setCursor(9, 1);
            lcd.print(Temp_Alarm);
            lcd.setCursor(14, 1);
            lcd.print((char)223);    //显示 o 符号
            lcd.setCursor(15, 1);
            lcd.print("C");          //显示字母 C
            if(alarm_choose == 1){
                Set_Alarm_Hour();
            }else if(alarm_choose == 2){
                Set_Alarm_Minute();
            }
        }
    }
}

```



```

        }else if(alarm_choose == 3){
            Set_Alarm_Temp();
        }else if(alarm_choose >= 4){
            alarm_choose = 0;
            break;
        }
    }
}

}

/** 正点蜂鸣，蜂鸣次数为小时数 */
void Point_Time_Alarm(){
    if(minute == 0 && second == 0){
        int times = hour % 12; // 12 小时制，0 点或 12 点响 12 下
        if(times == 0) times = 12;
        for(int i = 0; i < times; i++){
            tone(Tone, frequency0);
            delay(300);
            noTone(Tone);
            if(i < times - 1) delay(200); // 每次蜂鸣的间隔
        }
    }
}

/** 闹钟 指定时间蜂鸣 闹钟铃改为小星星 并显示 GET UP*/
void Clock_Alarm(){
    if(hour == alarm_hour && minute == alarm_minute && second == alarm_second){
        // LCD 显示 GET UP
        lcd.setCursor(0, 1);
        lcd.print("GET UP      ");

        // 第一句: 1 1 5 5 6 6 5
        tone(Tone, frequency1); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency1); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency5); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency5); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency6); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency6); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency5); delay(400); noTone(Tone); delay(100);
        // 第二句: 4 4 3 3 2 2 1
        tone(Tone, frequency4); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency4); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency3); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency3); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency2); delay(400); noTone(Tone); delay(50);
        tone(Tone, frequency2); delay(400); noTone(Tone); delay(50);
    }
}

```

```

        tone(Tone, frequency1); delay(400); noTone(Tone);

        // 恢复显示时间
        Display();
    }
}

/** 获取 LM35 温度 */
void GetTemperatures(){
    long a = analogRead(A3); // Get temperatures
    Temperatures = 500.0*a/1023.0;
    lcd.setCursor(9, 1) ;
    lcd.print(Temperatures); //获取温度
    lcd.setCursor(14, 1);
    lcd.print((char)223); //显示 o 符号
    lcd.setCursor(15, 1);
    lcd.print("C"); //显示字母 C
}

/** 超过指定温度报警 连续蜂鸣警报 同时显示 TOO HOT*/
void Temperatures_Alarm(){
    if(Temperatures >= Temp_Alarm){
        // LCD 显示 TOO HOT
        lcd.setCursor(0, 1);
        lcd.print("TOO HOT    ");

        // 循环响铃 25 次
        for(int i = 0; i < 25; i++){
            tone(Tone, frequency0); // 警报声
            delay(200);
            noTone(Tone);
            delay(300); // 间隔
        }
    }
}

void setup(){
    for(int i = 2; i <= 13; i++){
        pinMode(i, OUTPUT);
    }
    digitalWrite(add, HIGH);
    digitalWrite(minus, HIGH);
    digitalWrite(choose, HIGH);
    lcd.begin(16, 2); //初始化 LCD1602
    //通电读取 DS1302 芯片的时间
    rtc.writeProtect(false); // 关闭 DS1302 芯片写保护
    rtc.halt(false); //为 true 时 DS1302 暂停
}

```

```

    rtc.setTime(12, 0, 0);    // Set the time to 12:00:00 (24hr format)
    rtc.setDate(31,1,2024);  // Set the date to January 31th, 2024
}
void loop() {
    seconds = millis()/1000;    //获取单片机当前运行时间
    Display();                  //显示时间
    Set_Clock();                //设置时间
    Set_Alarm();                //设置闹钟
    Point_Time_Alarm();         //正点蜂鸣
    Clock_Alarm();              //闹钟时间蜂鸣
    GetTemperatures();          //获取 LM35 温度
    Temperatures_Alarm();       //超过指定温度报警
    //断电将单片机的当前时间写到 DS1302 芯片中
    Time t;
    t=rtc.getTime();
    rtc.setTime(t.hour,t.min,t.sec); //设置 DS1302 芯片初始时间
    rtc.setDate(t.date,t.mon,t.year);
    set(t.year,t.mon,t.date,t.hour,t.min,t.sec);
}

```

六、心得与体会

本次实验中，我完成了原理图绘制、PCB 图设计与相关程序编写，既锻炼了电路设计与软件操作能力，也加深了对电子工程实践流程的理解。

在原理图绘制阶段，我通过查阅资料、参考示例，进一步熟悉了 Altium Designer 的基本操作，理清了元件间的连接逻辑和引脚与封装的作用与重要性，避免后续步骤出现问题。在 PCB 图设计阶段，这一次更多的器件也进一步加强了我的整体布局排练能力和设计思维。在编程阶段，我学习并修改了参考代码，熟悉了 Arduino 的基础语法与控制逻辑，能够根据功能需求调整程序，解决了部分基础问题。

在整个设计过程中，我也遇到过不少困难，比如初期难以找到相应的元器件、对元器件的封装不熟悉、原理图连线出现混乱、PCB 图连线出现大量交叉导致后续连线困难、代码调试时逻辑功能难以正确实现等。不过，通过反复核对资料、请教老师同学、自身钻研等方法，这些困难最终都一一解决了。这次实验让我完整体验了从设计到实现的流程，不仅提升了我的实操能力，也培养了我的耐心与严谨的工作态度，为后续我的专业学习实验操作打下了良好基础。